

## ATIVIDADE

# Sistema de Controle de Compras com Cartão de Crédito

Programação Orientada a Objetos • Dart

## 1. Contexto

Uma instituição financeira deseja desenvolver um sistema simples para controlar as compras realizadas por meio de um cartão de crédito.

O sistema deverá permitir a criação de um cartão com limite definido, o registro de compras, a verificação de saldo disponível e a aprovação ou recusa das compras de acordo com o limite disponível.

A aplicação será desenvolvida em Dart, utilizando conceitos de Programação Orientada a Objetos — POO.

O objetivo da atividade é permitir que os alunos compreendam o problema, identifiquem as entidades envolvidas, definam as responsabilidades de cada classe e implementem uma solução organizada.

## 2. Objetivo da atividade

Desenvolver uma aplicação capaz de simular o funcionamento básico de um cartão de crédito.

O sistema deverá controlar:

- o limite total do cartão;
- o saldo disponível;
- as compras aprovadas;
- as compras recusadas;
- o valor total utilizado;
- a listagem das compras realizadas.

Durante o desenvolvimento, deverão ser aplicados conceitos como:

- classes e objetos;
- atributos e métodos;
- construtores;
- encapsulamento;
- validação de dados;
- composição entre classes;

- separação de responsabilidades;
- ordenação de objetos;
- tratamento de situações inválidas;
- testes automatizados.

O foco principal não será apenas fazer o sistema funcionar. A solução deverá apresentar uma estrutura organizada, na qual cada classe tenha responsabilidades bem definidas.

### 3. Descrição do problema

O sistema deverá representar um cartão de crédito que possui um limite financeiro definido no momento de sua criação.

Inicialmente, o saldo disponível do cartão será igual ao seu limite total. Sempre que uma compra for aprovada, o valor correspondente deverá ser descontado do saldo disponível.

Uma compra somente poderá ser aprovada quando o cartão possuir saldo suficiente para cobrir seu valor.

Caso o saldo disponível seja menor que o valor da compra, a operação deverá ser recusada. Nesse caso, o saldo do cartão não poderá ser alterado e a compra não deverá ser armazenada entre as compras aprovadas.

O sistema também deverá permitir a consulta das informações do cartão e a visualização das compras aprovadas em ordem crescente de valor.

### 4. Regras de negócio

1. Todo cartão deverá possuir um limite definido no momento de sua criação.
2. O limite do cartão deverá ser maior que zero.
3. O saldo disponível inicialmente deverá ser igual ao limite total do cartão.
4. Toda compra deverá possuir uma descrição e um valor.
5. A descrição da compra não poderá ser vazia.
6. O valor da compra deverá ser maior que zero.
7. Uma compra somente poderá ser aprovada quando seu valor for menor ou igual ao saldo disponível do cartão.
8. Quando uma compra for aprovada, ela deverá ser armazenada pelo sistema, o valor deverá ser descontado do saldo disponível e a operação deverá ser informada como aprovada.
9. Quando uma compra for recusada, ela não deverá ser armazenada entre as compras aprovadas, o saldo disponível deverá permanecer inalterado e o sistema deverá informar o motivo da recusa.
10. O sistema deverá permitir consultar o limite total do cartão.
11. O sistema deverá permitir consultar o saldo disponível.

12. O sistema deverá permitir consultar o valor total utilizado.
13. O sistema deverá permitir consultar a quantidade de compras aprovadas.
14. As compras aprovadas deverão ser apresentadas em ordem crescente de valor.
15. O saldo e a lista de compras não deverão ser alterados diretamente por outras partes da aplicação.

## 5. Entidades do sistema

### 5.1 Compra

A entidade Compra representa uma aquisição realizada pelo cliente.

Cada compra deverá possuir uma descrição e um valor.

A própria entidade deverá garantir que seus dados sejam válidos. Portanto, não deverá ser possível criar uma compra sem descrição ou com valor menor ou igual a zero.

A compra também deverá oferecer uma forma de ser comparada com outras compras pelo seu valor. Essa comparação será utilizada para ordenar a lista de compras em ordem crescente.

### 5.2 Cartão de crédito

A entidade Cartão de Crédito representa o cartão utilizado para realizar as compras.

Ela deverá ser responsável por:

- armazenar o limite total;
- controlar o saldo disponível;
- armazenar as compras aprovadas;
- verificar se existe saldo suficiente;
- aprovar compras;
- recusar compras;
- calcular o valor total utilizado;
- informar a quantidade de compras realizadas;
- fornecer a lista de compras ordenada.

O cartão deverá controlar internamente suas próprias informações. Outras partes do sistema não deverão modificar diretamente o saldo ou inserir compras na lista.

Essa responsabilidade é importante porque impede que o cartão entre em um estado inválido. Por exemplo, uma parte externa não deverá conseguir adicionar uma compra sem descontar seu valor do saldo.

### 5.3 Serviço de compras

A aplicação poderá possuir uma classe responsável por organizar o processo de realização de uma compra.

Esse serviço poderá ser responsável por receber os dados informados pelo usuário, criar uma nova compra, solicitar ao cartão a tentativa de aprovação e informar o resultado da operação.

A utilização de um serviço ajuda a evitar que a função principal da aplicação concentre todas as regras do sistema.

A classe principal deverá cuidar apenas da interação com o usuário e da organização do fluxo da aplicação, enquanto as classes de domínio deverão cuidar de suas próprias responsabilidades.

## **6. Encapsulamento**

O sistema deverá utilizar encapsulamento para proteger os dados internos das classes.

O saldo disponível, o limite e a lista de compras não deverão ser livremente modificados por qualquer parte do programa.

O cartão deverá oferecer operações próprias para realizar uma compra, consultar o saldo, consultar o limite, consultar as compras e calcular o valor utilizado.

Dessa forma, qualquer alteração no saldo deverá ocorrer somente quando uma compra for aprovada pelo próprio cartão.

O encapsulamento também contribui para a manutenção do sistema, pois as regras de alteração dos dados ficam concentradas em um único local.

## **7. Funcionamento esperado**

Ao iniciar a aplicação, deverá existir um cartão com um limite definido.

O usuário deverá poder informar os dados de uma compra. A aplicação deverá verificar se o valor é válido e se existe saldo suficiente para sua aprovação.

Quando houver saldo suficiente, a compra deverá ser aprovada e armazenada. O saldo disponível deverá ser reduzido pelo valor da compra.

Quando não houver saldo suficiente, a compra deverá ser recusada. O saldo deverá permanecer igual e a compra não deverá fazer parte da lista de compras aprovadas.

Ao consultar as informações do cartão, o sistema deverá apresentar o limite total, o valor utilizado, o saldo disponível e a quantidade de compras aprovadas.

Ao solicitar a listagem das compras, o sistema deverá apresentar somente as compras aprovadas, organizadas em ordem crescente de valor.

## **8. Exemplo de situação**

Considere um cartão com limite total de mil reais. Inicialmente, o cartão possuirá mil reais de saldo disponível.

O cliente realiza uma compra no valor de cem reais. Como existe saldo suficiente, a compra é aprovada e o saldo passa a ser de novecentos reais.

Em seguida, o cliente realiza uma compra no valor de seiscentos reais. A compra também é aprovada e o saldo passa a ser de trezentos reais.

Depois disso, o cliente tenta realizar uma compra no valor de mil e quinhentos reais. Como o saldo disponível é de apenas trezentos reais, a compra deverá ser recusada.

Após essas operações, o sistema deverá considerar apenas as duas primeiras compras como aprovadas. A compra recusada não deverá ser armazenada e o saldo deverá continuar sendo de trezentos reais.

## 9. Requisitos da implementação

- ser desenvolvida na linguagem Dart;
- possuir uma classe para representar uma compra;
- possuir uma classe para representar o cartão de crédito;
- utilizar construtores;
- utilizar atributos e métodos;
- aplicar encapsulamento;
- validar os dados recebidos;
- impedir compras com valor inválido;
- impedir cartões com limite inválido;
- aprovar compras somente quando houver saldo suficiente;
- armazenar somente compras aprovadas;
- manter o saldo correto após cada operação;
- permitir a consulta das informações do cartão;
- listar as compras em ordem crescente de valor;
- evitar que toda a lógica fique concentrada na função principal;
- utilizar nomes claros para classes, atributos, métodos e variáveis;
- evitar duplicação de código;
- tratar adequadamente as situações inválidas;
- possuir testes automatizados para as principais regras.

## 10. Testes que deverão ser realizados

### 10.1 Criação do cartão

Verificar se o cartão é criado corretamente quando recebe um limite válido. Também deverá ser verificado se o saldo inicial é igual ao limite informado.

### **10.2 Limite inválido**

Verificar o comportamento do sistema quando o cartão recebe limite igual a zero ou limite negativo. O sistema deverá impedir a criação do cartão ou tratar a situação de maneira adequada e consistente.

### **10.3 Criação de uma compra válida**

Verificar se é possível criar uma compra quando a descrição e o valor são válidos.

### **10.4 Compra com dados inválidos**

Verificar o comportamento do sistema quando a compra possui descrição vazia, valor igual a zero ou valor negativo. A compra inválida deverá ser impedida ou rejeitada pelo sistema.

### **10.5 Compra aprovada**

Criar um cartão com saldo suficiente e realizar uma compra válida. O teste deverá verificar se a compra foi aprovada, se o saldo foi reduzido corretamente, se a compra foi armazenada, se o valor total utilizado foi atualizado e se a quantidade de compras foi atualizada.

### **10.6 Compra recusada**

Criar uma compra com valor superior ao saldo disponível. O teste deverá verificar se a compra foi recusada, se o saldo permaneceu inalterado, se a compra não foi armazenada, se o valor total utilizado não foi alterado e se a quantidade de compras aprovadas permaneceu a mesma.

### **10.7 Ordenação das compras**

Realizar diferentes compras aprovadas com valores variados. Em seguida, verificar se a listagem apresenta as compras em ordem crescente de valor, independentemente da ordem em que foram realizadas.

## **11. Organização recomendada**

O projeto deverá ser organizado de maneira que cada parte do sistema possua uma responsabilidade clara.

A classe responsável pela compra deverá cuidar dos dados e das validações da compra.

A classe responsável pelo cartão deverá controlar o limite, o saldo e as compras aprovadas.

Uma classe de serviço poderá organizar o processo de realização das compras.

A função principal deverá ser utilizada principalmente para iniciar a aplicação, receber informações e apresentar resultados, evitando concentrar nela todas as regras de negócio.

A organização dos arquivos poderá seguir uma estrutura baseada em modelos, serviços e testes. Entretanto, os alunos poderão propor outra organização, desde que ela seja clara e justificada.

## 12. Critérios de avaliação

### Modelagem das entidades

Será avaliada a capacidade de identificar e representar corretamente as entidades do problema.

### Aplicação dos conceitos de POO

Serão observados o uso adequado de classes, a criação de objetos, a utilização de construtores, o encapsulamento, a separação de responsabilidades e o relacionamento entre as classes.

### Implementação das regras de negócio

Será verificado se o sistema aprova compras válidas, recusa compras sem saldo suficiente, atualiza o saldo corretamente, armazena somente compras aprovadas e mantém os dados consistentes.

### Validação e tratamento de erros

Será avaliado o tratamento de limites, descrições e valores inválidos.

### Ordenação e consultas

Será verificado se as compras são listadas em ordem crescente de valor e se os dados do cartão são apresentados corretamente.

### Testes automatizados

Serão avaliados a existência, a clareza e a abrangência dos testes.

### Qualidade do código

Serão considerados legibilidade, organização, nomes significativos, ausência de duplicação, clareza das responsabilidades e facilidade de manutenção.

## 13. Entrega esperada

A entrega deverá conter o projeto desenvolvido em Dart, as classes responsáveis pelo cartão e pelas compras, os testes automatizados, as validações necessárias, a aplicação funcionando de acordo com as regras descritas e uma breve explicação sobre a responsabilidade de cada classe.

A explicação deverá apresentar como o encapsulamento foi utilizado e de que maneira as regras de negócio foram distribuídas entre as classes.

## 14. Desafios opcionais

- criação de um menu interativo;
- consulta de compras por descrição;
- filtro de compras por valor;
- registro da data de cada compra;

- classificação das compras por categoria;
- cancelamento ou estorno de uma compra;
- geração de uma fatura;
- controle de compras parceladas;
- criação de diferentes tipos de cartão;
- utilização de classes abstratas;
- utilização de interfaces;
- armazenamento das informações em arquivos.

As funcionalidades opcionais deverão ser implementadas sem comprometer as regras principais do sistema.

## 15. Perguntas para reflexão

16. Qual classe deve ser responsável por controlar o saldo do cartão?
17. Por que o saldo não deve ser alterado diretamente pela aplicação principal?
18. O que deve acontecer quando uma compra não possui saldo suficiente?
19. Por que a própria classe Compra deve validar seus dados?
20. Qual é a vantagem de separar a interação com o usuário das regras de negócio?
21. Como o sistema poderia ser adaptado para trabalhar com diferentes tipos de cartão?
22. Quais responsabilidades pertencem ao cartão e quais pertencem à compra?
23. O que poderia acontecer se a lista de compras pudesse ser alterada diretamente por qualquer parte do sistema?
24. Como os testes ajudam a garantir que o saldo está sendo atualizado corretamente?
25. Quais partes dessa solução poderiam ser reutilizadas em um sistema real?

## 16. Desafio principal

Implemente o sistema de controle de compras seguindo as regras apresentadas.

A estrutura interna das classes e a organização do projeto deverão ser definidas pelos alunos. As decisões deverão ser justificadas com base nos princípios da Programação Orientada a Objetos.

O sistema deverá funcionar corretamente, mas também deverá apresentar uma organização que facilite sua compreensão, manutenção e evolução.